Ritmo: Senior Project

August 1

2014

Michelle Duffy

Charleston Southern University

About the Project

Purpose:

Ritmo (meaning "rhythm" in Spanish) is a Windows Form program written in C# designed to play audio by utilizing the nAudio library. It was created using Microsoft Visual Studio 2010 on a computer running Windows Vista and it was tested on a computer running Windows 7. The goal of this project was to create a program outside the classroom setting and develop it through the entire process from concept to completion while learning and using an open-source library.

Difficulties:

Creating the user interface was surprisingly difficult. Between the limitations of a Windows Form and wanting my program to look like other programs and apps on the market, it was difficult to decide which images to use for the background and buttons. There were a number of free PSD kits online but I don't have PhotoShop nor do I have experience creating or manipulating images for graphic design.

Making the form movable was a difficulty that I did not overcome, however, that was more of an issue of time. I felt that other functionality was more important so I put being able to move the form off until the end but then I ran out of time before I could fix it.

Another challenge that I came across was to keep songs from overlapping each other. At one point, if you played a song and then played a different song without actually hitting the Stop button first, the original song would continue and the new song would start to play on top

of it. For a while I was stumped because it seemed that using WaveOut.Stop() should work but eventually I decided to try disposing of the WaveOut and creating a new instance of it for the new song, which finally solved the problem.

Possible Future Improvements:

-Implement the ability to create and store multiple playlists.

-Add a seek-bar to show position within the song and allow user to choose exact (to the second) spot.

-Convert to mobile app.

-Display album art.

-Enable scrolling for the songNameLabel instead of having a long string of text go over the background image.

-Include better error handling and error messages.

Creation of the User Interface:

The background image was found by searching online for images related to audio and music. The current one was chosen because it had a sense of the days when radio first became commonly used, when it was the "latest and greatest thing." Another reason is simply because the headphones represent listening to music and the microphone represents recording audio, which could be a feature added in later. Once a suitable image was selected, it was downloaded and then imported into the project resources. The original image turned out to be too large so it was resized using Paint. The image was then set as the background using the properties toolbar for the form. To give the program a more unique look, the border was

removed and to avoid having the background image stretched the resizing options were set to false.

The button images were found by searching www.deviantart.com for audio controls. The icon set being used was found at http://0dd0ne.deviantart.com/art/300-Black-and-White-iCONS-144272753, however they had to be resized which was done using the website www.picresize.com. However, the icon set didn't include icons that could be used to represent fast forward and rewind so the previous and next icons had to be reused and modified using Paint. The images were then imported into the project resources and individually set as the background for a PictureBox control, with the BackColor being set to transparent.

Source Code

(Not including Visual Studio generated code.)

```
1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel;
4  using System.Data;
5  using System.Drawing;
6  using System.Linq;
7  using System.Text;
8  using System.Windows.Forms;
9  using System.IO;
10
11 using NAudio;
12 using NAudio.Wave;
13
14 namespace Ritmo
15 {
16   public partial class RitmoPlayer : Form
17   {
18     // For playing the audio.
19     private IWavePlayer waveOutDevice;
20
21     // For reading the audio file.
22     private AudioFileReader audioFileReader;
23
24     // For writing to SongPlaylist.txt.
25     private StreamWriter fileWriter;
26
27     // For reading SongPlaylist.txt.
28     private StreamReader fileReader;
29
30     // Path for SongPlaylist.txt
31     private string filePath;
32
33     // Path of the song.
34     private string songPath;
35
36     // Name of the song.
37     private string songName;
38
39     // Path of the currently playing song.
40     private string curPlayingSongPath;
41
42     // Path of the next song to be played.
43     private string selectedNextSongPath;
44
45     // Index of the selected song in the playlist.
46     private int selectedPlaylistIndex;
47
48     // List of the paths of each song file in the playlist.
49     private List<string> playlistLocations;
50
51     // List of the song titles in the playlist.
52     private List<string> playlistTitles;
```

```
53
54      // The most recent index selected by the user.
55      private int userRecentIndex;
56
57      /**
58       * Constructor
59       **/
60      public RitmoPlayer()
61      {
62        // Initialize, set up the new WaveOut, and set up the tick for the timer.
63        InitializeComponent();
64        waveOutDevice = new WaveOut();
65        this.timer1.Interval = 250;
66        this.timer1.Tick += new EventHandler(timer1_Tick);
67
68        // Create a list for the song file paths and the song titles.
69        playlistLocations = new List<string>();
70        playlistTitles = new List<string>();
71
72        // Set the path to SongPlaylist.txt and start reading from it.
73        filePath = @"..\..\Resources\SongPlaylist.txt";
74        fileReader = new System.IO.StreamReader(filePath);
75        string line = fileReader.ReadLine();
76
77        if (line != "")
78        {
79          // If the file isn't empty, set the user's most recent index from the first
          line
80          // in the file.
81          userRecentIndex = Convert.ToInt32(line);
82
83          // Until reach the end of the file, read each line and alternate between adding
           the
84          // paths to the list of song locations and the titles to the list of song
          titles.
85          while (!fileReader.EndOfStream)
86          {
87            string fileLine = fileReader.ReadLine();
88            playlistLocations.Add(fileLine);
89            playlistTitles.Add(fileReader.ReadLine());
90          }
91
92          // If the count of the total locations in the list is greater than zero, then
          set the
93          // path to the user's most recent index and update the song name label with the
           title
94          // of that song from the list.
95          if (playlistLocations.Count > 0)
96          {
97            songPath = playlistLocations[userRecentIndex];
98            songNameLabel.Text = playlistTitles[userRecentIndex];
99          }
```

```
100         }
101
102         // Otherwise, if the file was empty, just set the user's index to zero and close
            the file.
103         else
104           userRecentIndex = 0;
105
106         fileReader.Close();
107
108         // Connect tool tips to their respective controls and set them to visible.
109         openToolTip.ShowAlways = true;
110         openToolTip.SetToolTip(openButton, "Open the file explorer to choose a song.");
111         playToolTip.ShowAlways = true;
112         playToolTip.SetToolTip(playButton, "Begin playing the selected song.");
113         pauseToolTip.ShowAlways = true;
114         pauseToolTip.SetToolTip(pauseButton, "Pause the currently playing song.");
115         stopToolTip.ShowAlways = true;
116         stopToolTip.SetToolTip(stopButton, "Stop the currently playing song.");
117         rewindToolTip.ShowAlways = true;
118         rewindToolTip.SetToolTip(rewindButton, "Rewind the currently playing song.");
119         fastForwardToolTip.ShowAlways = true;
120         fastForwardToolTip.SetToolTip(fastForwardButton, "Fast forward the currently
            playing song.");
121         removeToolTip.ShowAlways = true;
122         removeToolTip.SetToolTip(removeFromPlaylistButton, "Remove the selected song from
             the playlist.");
123       }
124
125   /* Form Events */
126       private void RitmoPlayer_Load(object sender, EventArgs e)
127       {
128         // Loop through the list of song titles and add each one to the playlist listBox.
129         foreach (var item in playlistTitles)
130         {
131           if (item != null)
132             playlistBox.Items.Add(item);
133           else
134             break;
135         }
136
137         // If the listBox isn't empty, go ahead and select whichever one the user last
            selected.
138         if (playlistBox.Items.Count > 0)
139           playlistBox.SetSelected(userRecentIndex, true);
140       }
141
142       private void closeButton_Click(object sender, EventArgs e)
143       {
144         fileWriter = new System.IO.StreamWriter(filePath);
145
146         // Write the user's most recently used index to SongPlaylist.txt.
147         fileWriter.WriteLine(userRecentIndex);
```

```
148
149         // Loop through the items in the playlist listBox and save the corresponding
            paths to the file
150         // along with the song titles.
151         int index = 0;
152         foreach (var item in playlistBox.Items)
153         {
154           fileWriter.WriteLine(playlistLocations[index]);
155           fileWriter.WriteLine(item.ToString());
156           index++;
157         }
158
159         // Close the file and the form.
160         fileWriter.Close();
161         this.Close();
162       }
163
164   /* Control Events */
165       private void openButton_Click(object sender, EventArgs e)
166       {
167         // Get the path of the song that the user chose.
168         songPath = SelectInputFile();
169         if (songPath != null)
170         {
171           audioFileReader = new AudioFileReader(songPath);
172           songStatusLabel.Text = "Click Play to begin listening.";
173
174           // Add the path to the list of locations. Get the song title from the path and
              add it to the
175           // list of titles and the playlist listBox.
176           getTitleFromPath();
177           playlistLocations.Add(songPath);
178           playlistTitles.Add(songName);
179           playlistBox.Items.Add(songName);
180
181           // Set the user's index to the last item in the playlist and select that song
              in the listBox.
182           userRecentIndex = (playlistLocations.Count - 1);
183           playlistBox.SetSelected(userRecentIndex, true);
184           selectedNextSongPath = songPath;
185         }
186       }
187
188       void playAudio()
189       {
190         if (waveOutDevice != null)
191         {
192           // A new song can be chosen when a song is playing and when it is paused so
              need to do
193           // separate checks.
194           if (waveOutDevice.PlaybackState == PlaybackState.Playing)
195           {
```

```
196             // Check if the currently playing song is the same as the chosen song.
197             // If they're different, then need to dispose of old WaveOut before
198             // initializing a new one in order to avoid overlapping songs.
199             if (songPath != curPlayingSongPath)
200             {
201               waveOutDevice.Stop();
202               waveOutDevice.Dispose();
203               waveOutDevice = new WaveOut();
204             }
205             else
206               return;
207           }
208           else if (waveOutDevice.PlaybackState == PlaybackState.Paused)
209           {
210             // Check if the currently playing song is the same as the chosen song.
211             // If they're the same, continue playing the song, update the status label,
212             // and re-enable the timer.
213             if (songPath == curPlayingSongPath)
214             {
215               waveOutDevice.Play();
216               timer1.Enabled = true;
217               songStatusLabel.Text = "Now playing:";
218               return;
219             }
220             // If they're different, then need to dispose of old WaveOut before
221             // initializing a new one in order to avoid overlapping songs.
222             else
223             {
224               waveOutDevice.Stop();
225               waveOutDevice.Dispose();
226               waveOutDevice = new WaveOut();
227             }
228           }
229         }
230
231         // If the song path is valid, dispose the audioFileReader in order to initialize
          a new one
232         // to use in the new WaveOut.
233         if (songPath != null)
234         {
235           audioFileReader.Dispose();
236           audioFileReader = new AudioFileReader(songPath);
237           waveOutDevice.Init(audioFileReader);
238           waveOutDevice.Play();
239
240           // Set the currently playing song path to the selected song path, update the
            status label,
241           // and enable the timer.
242           curPlayingSongPath = songPath;
243           timer1.Enabled = true;
244           songStatusLabel.Text = "Now playing:";
245         }
```

```
246         else
247         {
248           // Let the user know that they need to choose a song.
249           songStatusLabel.ForeColor = Color.Red;
250           songStatusLabel.Text = "Please select a song by clicking the Open button.";
251         }
252       }
253
254       private void playButton_Click(object sender, EventArgs e)
255       {
256         playAudio();
257       }
258
259       private void pauseButton_Click(object sender, EventArgs e)
260       {
261         if (waveOutDevice != null)
262         {
263           // If a song is playing, pause it, update the status label, and disable the
              timer.
264           if (waveOutDevice.PlaybackState == PlaybackState.Playing)
265           {
266             waveOutDevice.Pause();
267             songStatusLabel.Text = "Paused.";
268             timer1.Enabled = false;
269           }
270         }
271       }
272
273       private void stopButton_Click(object sender, EventArgs e)
274       {
275         // Stop the song playback, update the status label, and disable the timer.
276         waveOutDevice.Stop();
277         songStatusLabel.Text = "Stopped.";
278         timer1.Enabled = false;
279       }
280
281       private void rewindButton_Click(object sender, EventArgs e)
282       {
283         // Calculate new position
284         long newPos = audioFileReader.Position - (long)
            (audioFileReader.WaveFormat.AverageBytesPerSecond * 5.0);
285
286         // Set position
287         audioFileReader.Position = setPosition(newPos);
288       }
289
290       private void fastForwardButton_Click(object sender, EventArgs e)
291       {
292         // Calculate new position
293         long newPos = audioFileReader.Position + (long)
            (audioFileReader.WaveFormat.AverageBytesPerSecond * 5.0);
294
```

```
295         // Set position
296         audioFileReader.Position = setPosition(newPos);
297       }
298
299       private void previousButton_Click(object sender, EventArgs e)
300       {
301         // Set the selected index to the one the user chose in the listBox.
302         selectedPlaylistIndex = playlistBox.SelectedIndex;
303
304         // If the index is zero, there is no previous song so just return. Otherwise,
          decrement
305         // the selected index by 1, make the necessary updates, and set the selected
          index
306         // in the listBox.
307         if (selectedPlaylistIndex <= 0)
308           return;
309         else
310         {
311           selectedPlaylistIndex--;
312           updateWhenIndexChanges();
313           playlistBox.SetSelected(selectedPlaylistIndex, true);
314         }
315       }
316
317       private void nextButton_Click(object sender, EventArgs e)
318       {
319         // Set the selected index to the one the user chose in the listBox.
320         selectedPlaylistIndex = playlistBox.SelectedIndex;
321
322         // If the index is greater than the total number of items in the list of titles,
          there
323         // is no next song so just return. Otherwise, increment the selected index by 1,
          make the
324         // necessary updates, and set the selected index in the listBox.
325         if (selectedPlaylistIndex >= (playlistTitles.Count - 1))
326           return;
327         else
328         {
329           selectedPlaylistIndex++;
330           updateWhenIndexChanges();
331           playlistBox.SetSelected(selectedPlaylistIndex, true);
332         }
333       }
334
335       private void playlistBox_SelectedIndexChanged(object sender, EventArgs e)
336       {
337         // Get the currently selected item in the ListBox.
338         selectedPlaylistIndex = playlistBox.SelectedIndex;
339
340         // If the index is valid, make the necessary updates and get the title from the
          song path.
341         if (selectedPlaylistIndex >= 0)
```

```
342         {
343           updateWhenIndexChanges();
344           getTitleFromPath();
345         }
346       }
347
348       private void removeFromPlaylistButton_Click(object sender, EventArgs e)
349       {
350         // Remove the item from the list of locations and from the playlist listBox.
351         playlistLocations.RemoveAt(selectedPlaylistIndex);
352         playlistBox.Items.RemoveAt(selectedPlaylistIndex);
353
354         // Update the user's index but don't let it become negative.
355         if (userRecentIndex == 0)
356           userRecentIndex--;
357       }
358
359       private void playlistBox_MouseDoubleClick(object sender, MouseEventArgs e)
360       {
361         // Get the selected index from the playlist listBox.
362         selectedPlaylistIndex = playlistBox.SelectedIndex;
363
364         // If the index is valid, update the user's index, get the path from the list of
365         // locations, create a new audioFileReader using the path, and update the status
          label.
366         // then get the title from the path and call the playAudio function.
367         if (selectedPlaylistIndex >= 0)
368         {
369           userRecentIndex = selectedPlaylistIndex;
370           songPath = playlistLocations[selectedPlaylistIndex];
371           audioFileReader = new AudioFileReader(songPath);
372           songStatusLabel.Text = "Click Play to begin listening.";
373
374           getTitleFromPath();
375         }
376         playAudio();
377       }
378
379   /* Helper Functions */
380       private static string FormatTimeSpan(TimeSpan ts)
381       {
382         return string.Format("{0:D2}:{1:D2}", (int)ts.TotalMinutes, ts.Seconds);
383       }
384
385       void timer1_Tick(object sender, EventArgs e)
386       {
387         if (audioFileReader != null)
388         {
389           // Update the time labels with the respective times from the audioFileReader.
390           labelNowTime.Text = FormatTimeSpan(audioFileReader.CurrentTime);
391           labelTotalTime.Text = FormatTimeSpan(audioFileReader.TotalTime);
392         }
```

```
393     }
394
395     private static string SelectInputFile()
396     {
397       // Create a new OpenFileDialog that only accepts .mp3, .wav, and .aiff formats.
398       var ofd = new OpenFileDialog();
399       ofd.Filter = "Audio Files|*.mp3;*.wav;*.aiff";
400
401       // If valid, return the fileName from the OpenFileDialog.
402       if (ofd.ShowDialog() == DialogResult.OK)
403         return ofd.FileName;
404
405       return null;
406     }
407
408     private void getTitleFromPath()
409     {
410       // The last part of the path will be the song title so set the label to that
          substring.
411       int indexOfSlash = songPath.LastIndexOf('\\');
412       songName = songPath.Substring((indexOfSlash + 1));
413       songNameLabel.Text = songName;
414     }
415
416     private long setPosition(long newPos)
417     {
418       // Force it to align to a block boundary
419       if ((newPos % audioFileReader.WaveFormat.BlockAlign) != 0)
420         newPos -= newPos % audioFileReader.WaveFormat.BlockAlign;
421
422       // Force new position into valid range
423       newPos = Math.Max(0, Math.Min(audioFileReader.Length, newPos));
424
425       return newPos;
426     }
427
428     private void updateWhenIndexChanges()
429     {
430       // Update the user's index to the selected index.
431       userRecentIndex = selectedPlaylistIndex;
432
433       // Get the song path from the list of locations and initialize a new
          audioFileReader
434       // with that path. Then update the status label and get the title from the path.
435       songPath = playlistLocations[selectedPlaylistIndex];
436       audioFileReader = new AudioFileReader(songPath);
437       songStatusLabel.Text = "Click Play to begin listening.";
438       getTitleFromPath();
439     }
440
441 /* Clean up resources */
442     /**
```

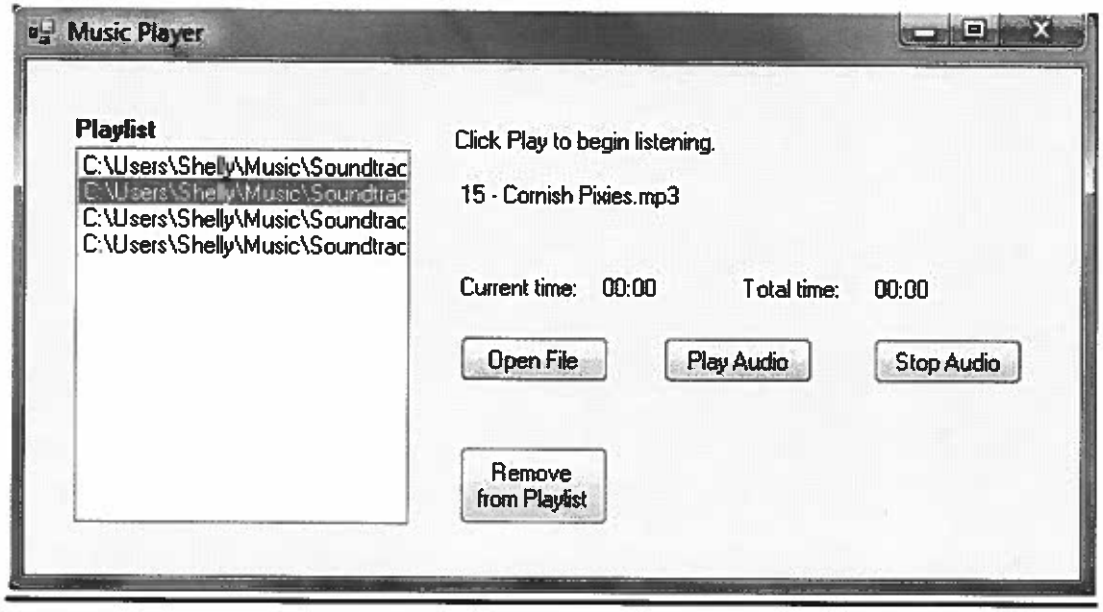
```
443         * Deconstructor
444         **/
445        private void CloseWaveOut()
446        {
447          if (waveOutDevice != null)
448            waveOutDevice.Stop();
449          if (audioFileReader != null)
450          {
451            audioFileReader.Dispose();
452            audioFileReader = null;
453          }
454          if (waveOutDevice != null)
455          {
456            waveOutDevice.Dispose();
457            waveOutDevice = null;
458          }
459        }
460      }
461    }
462
```

Ritmo, Version 1

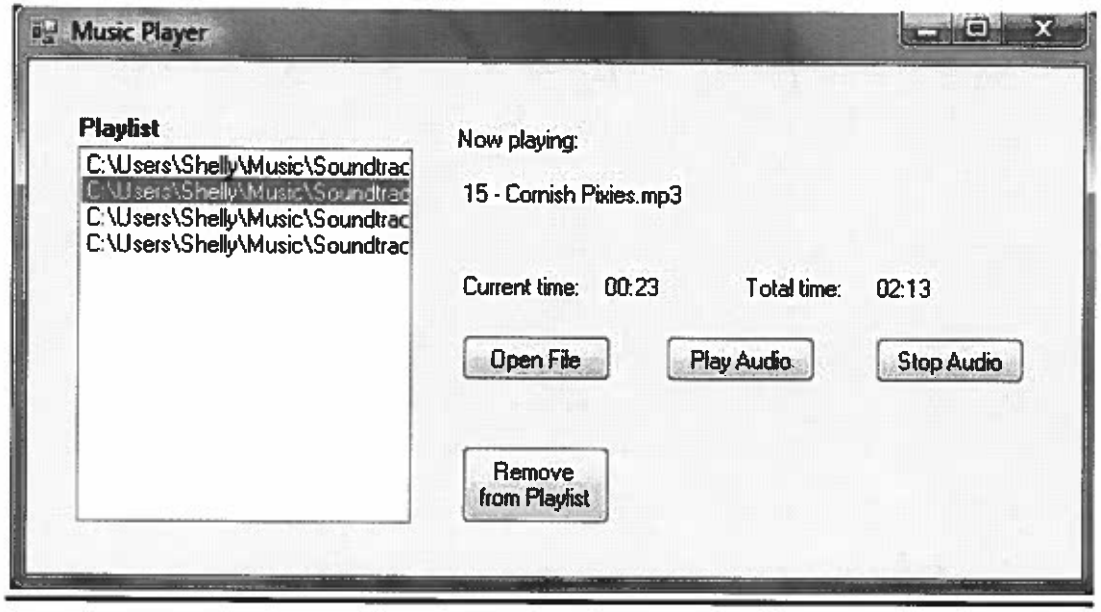
Screenshots

Player on open

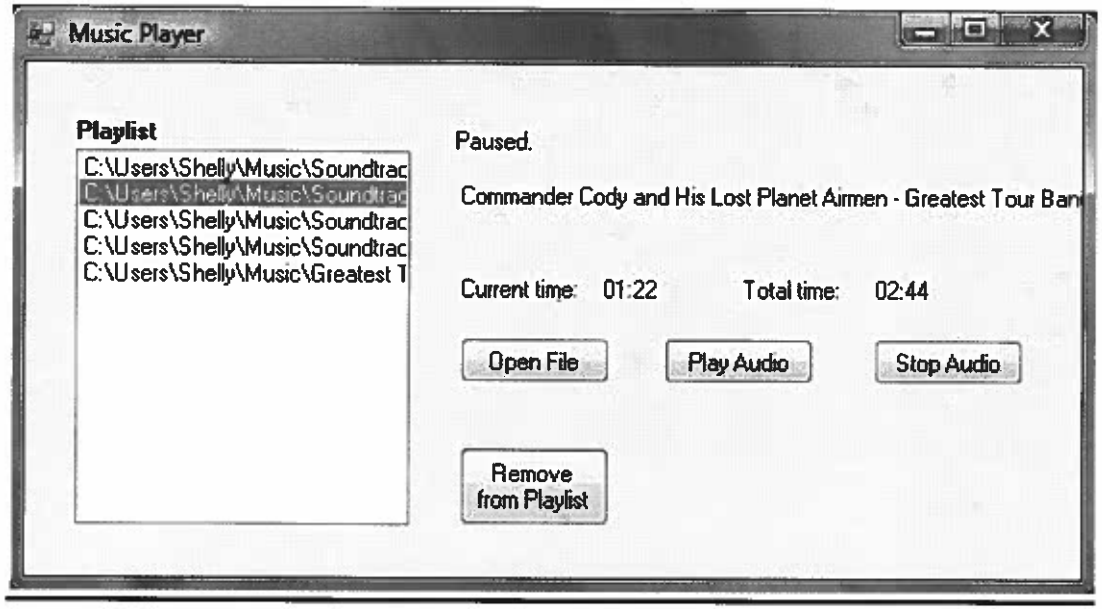
Select a song in the playlist



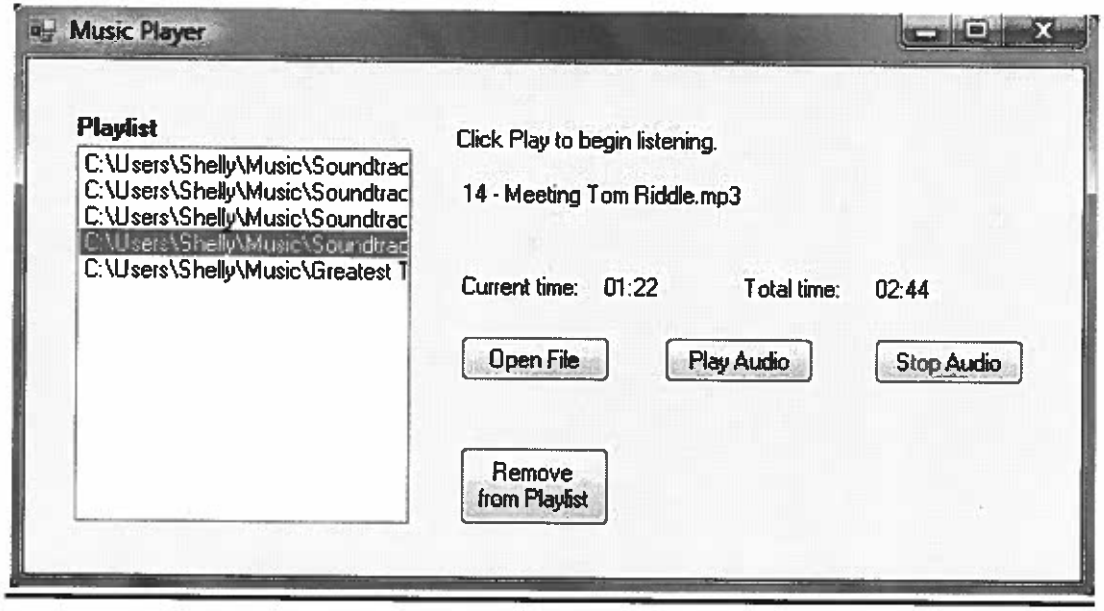
Playing a song from the playlist



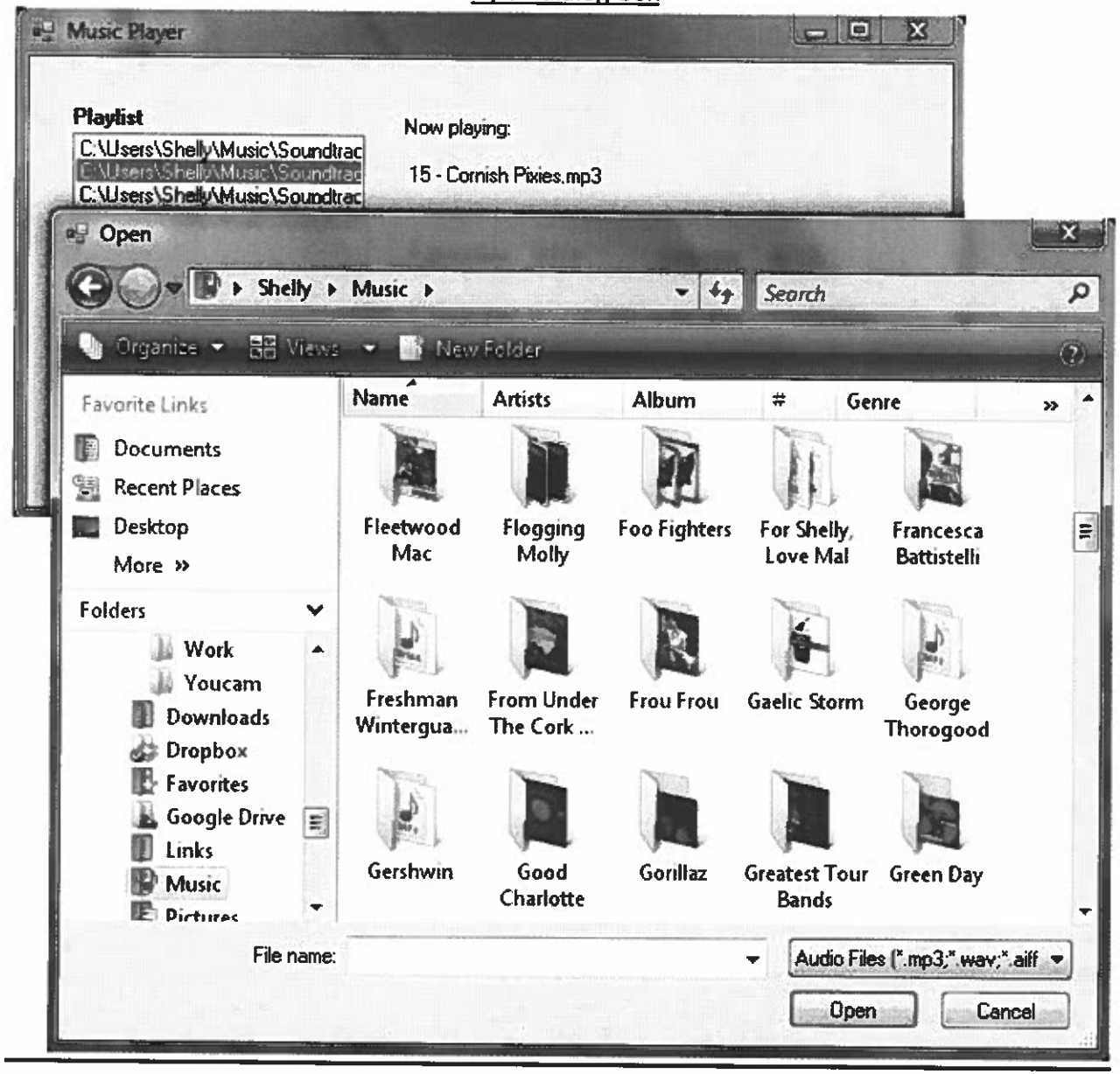
Song is stopped



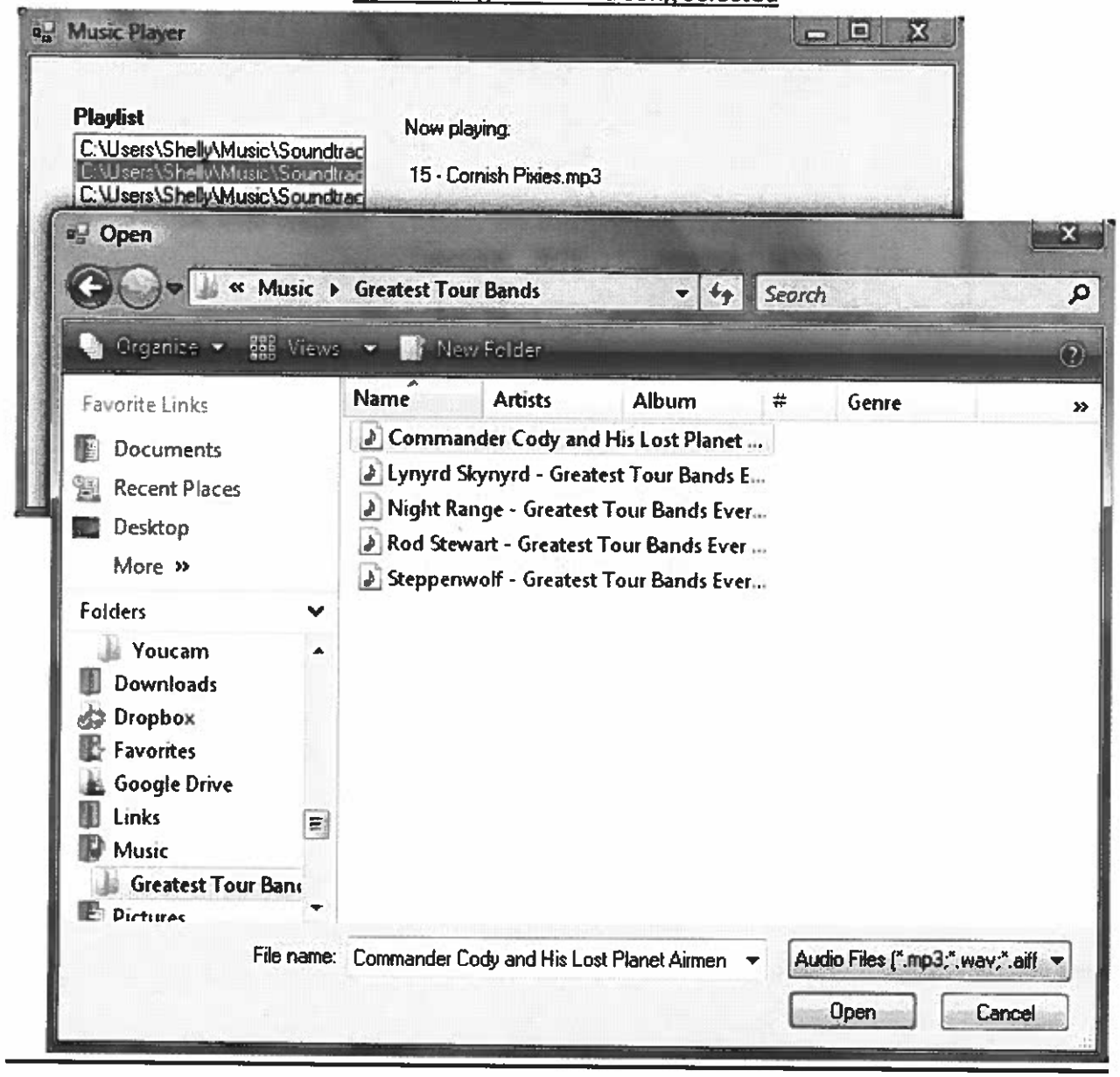
Selected song to be removed from the playlist



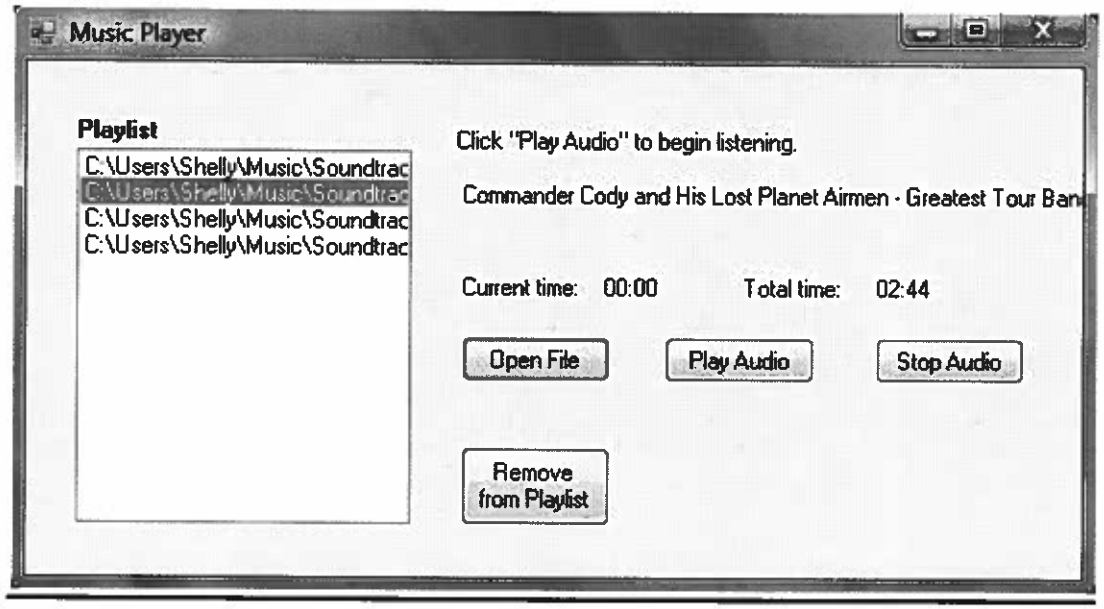
Open dialog-box



Open dialog-box with a song selected



Song from open dialog box is ready to play



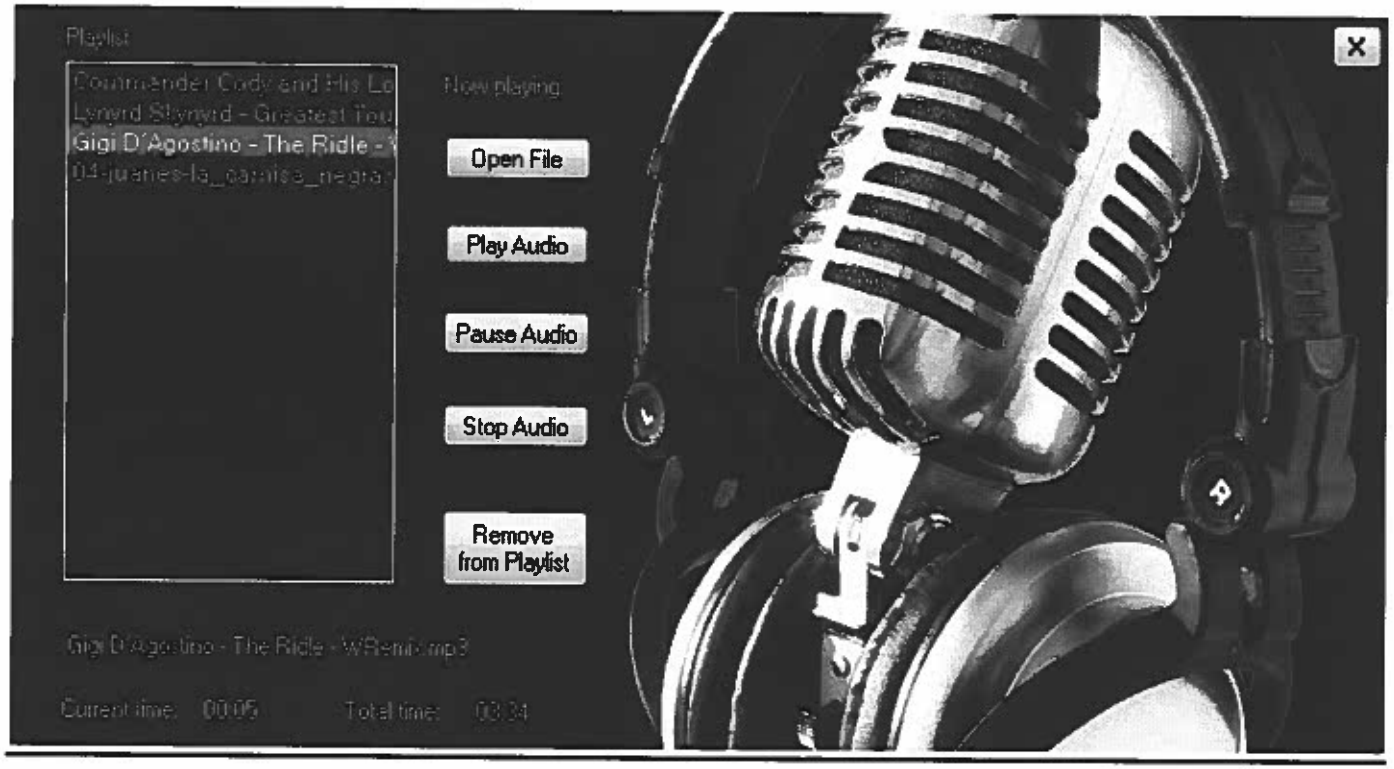
Playing the song from open dialog box

Ritmo, Version 2

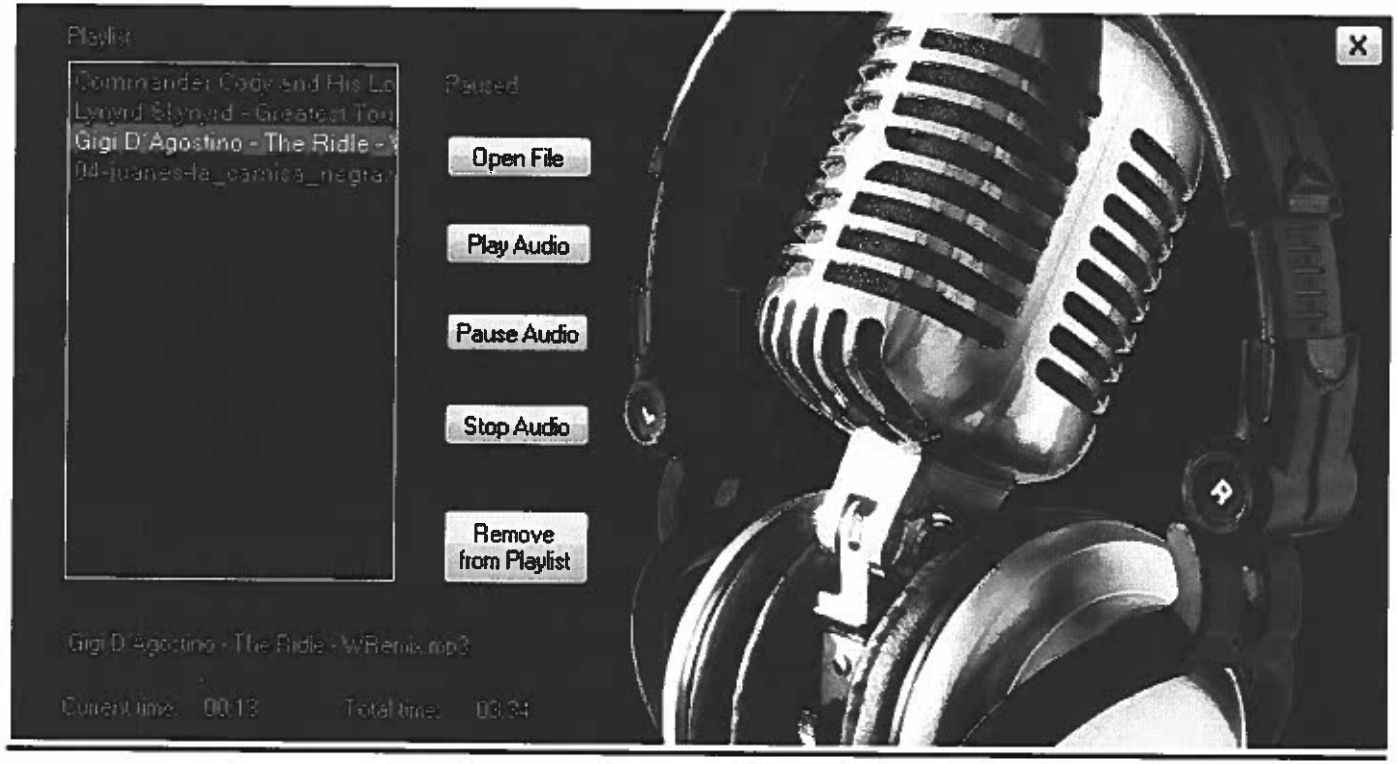
Screenshots

Player on open

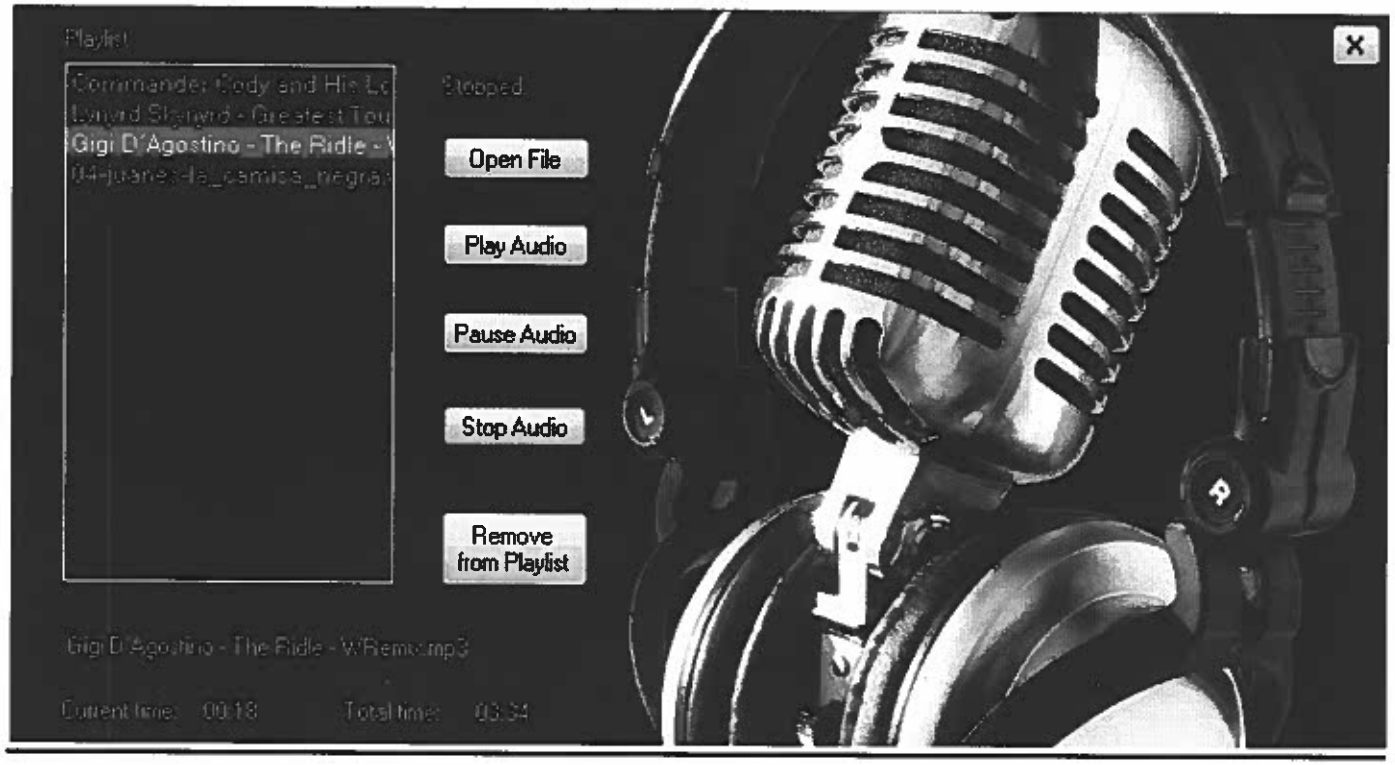
Playing a song from the playlist



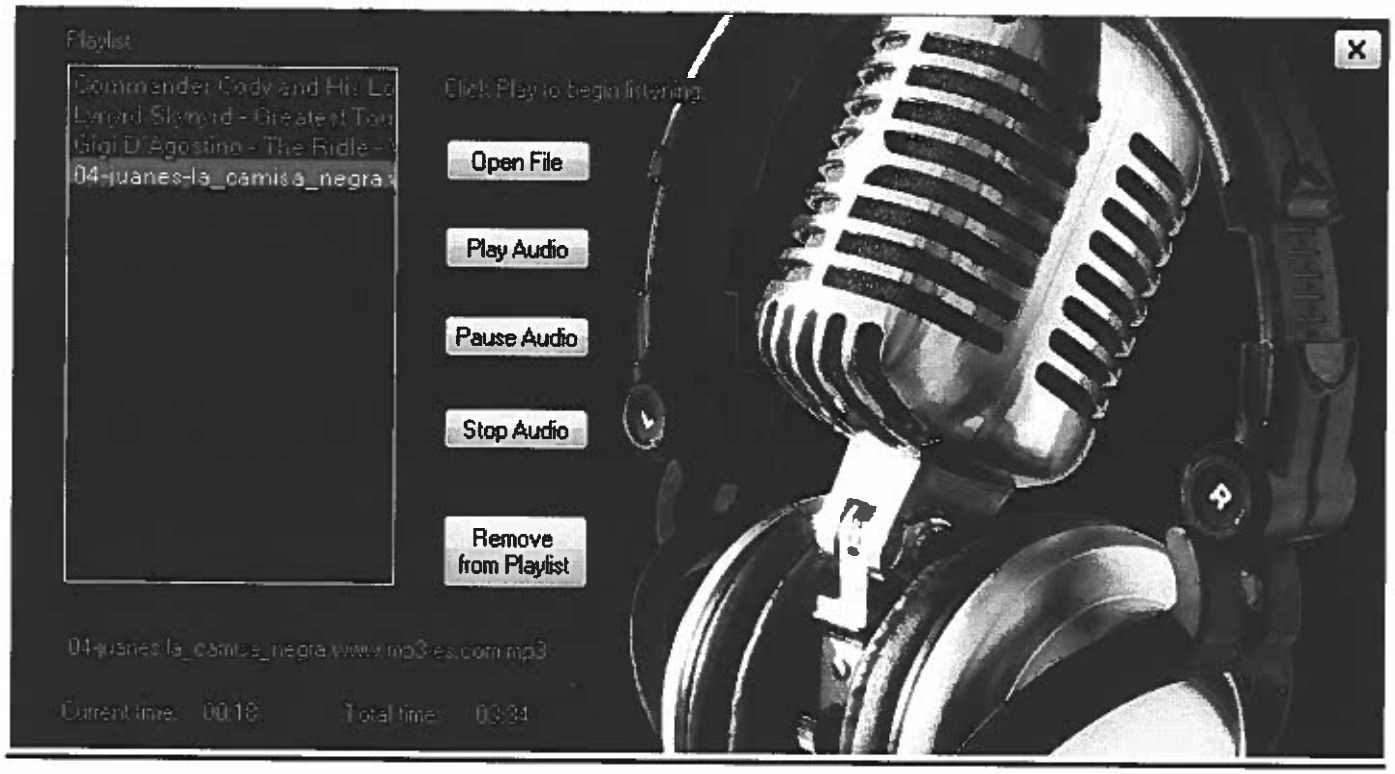
Song is paused



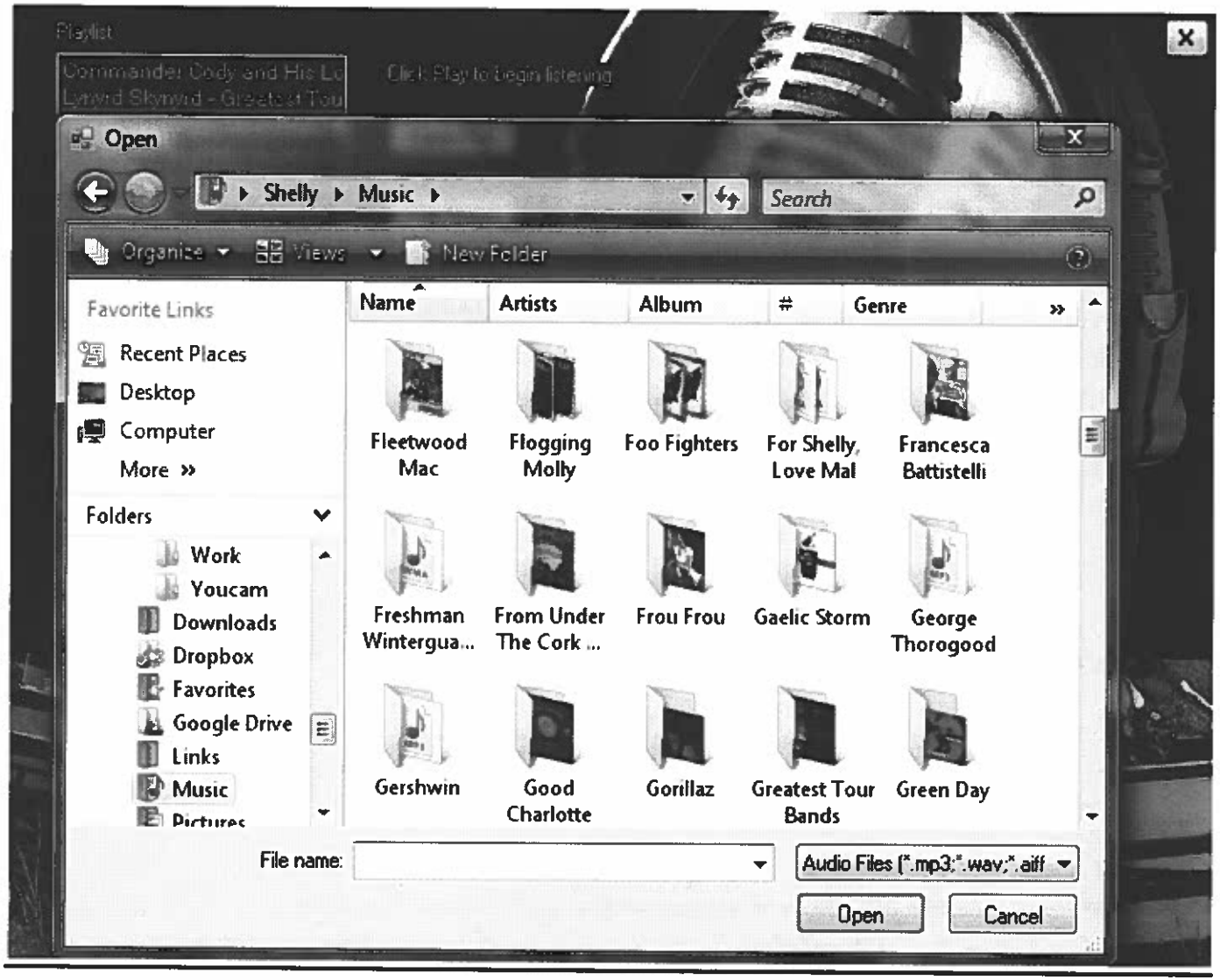
Song is stopped



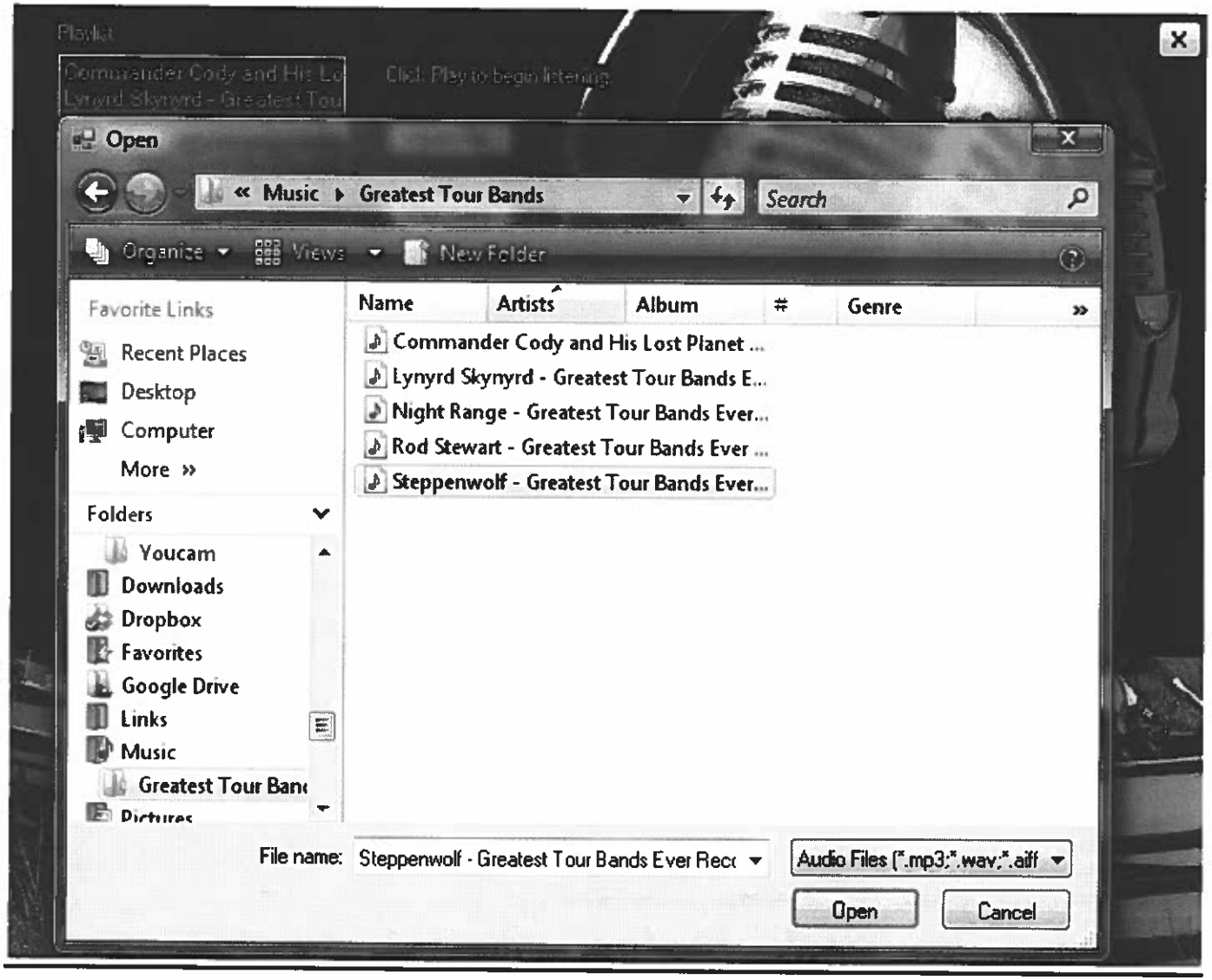
Selected song to be removed from the playlist



Open dialog-box



Open dialog-box with a song selected

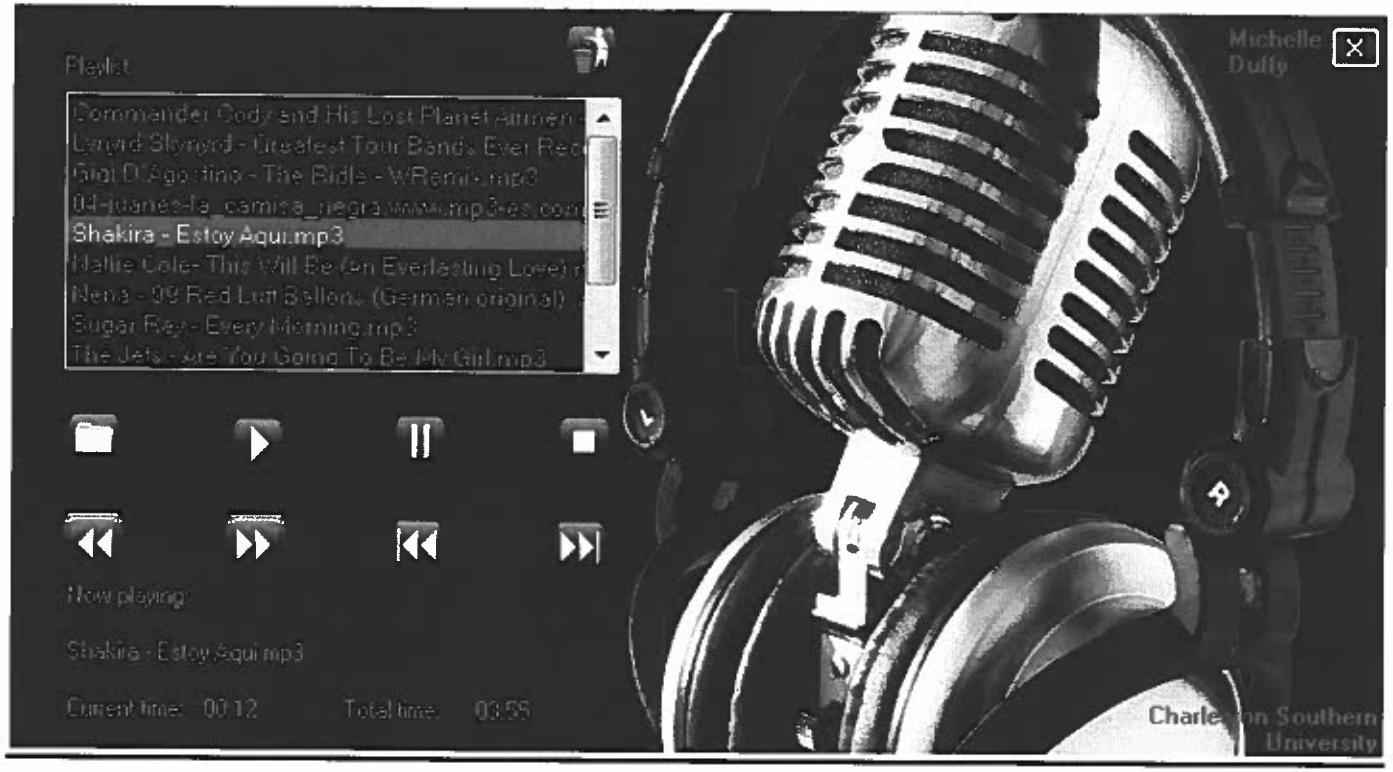


Ritmo, Version 3

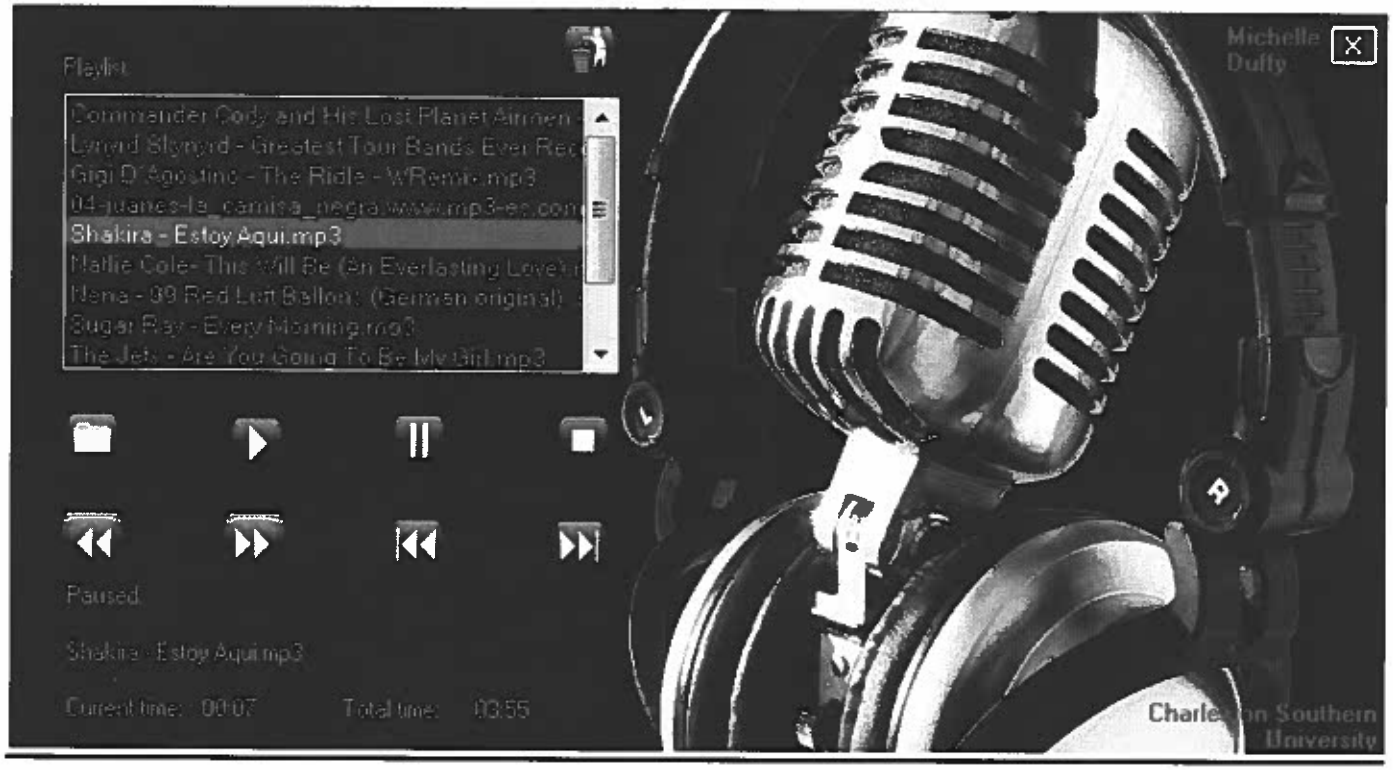
Screenshots

Player on open with a full playlist

Playing a song from the playlist



Song is paused

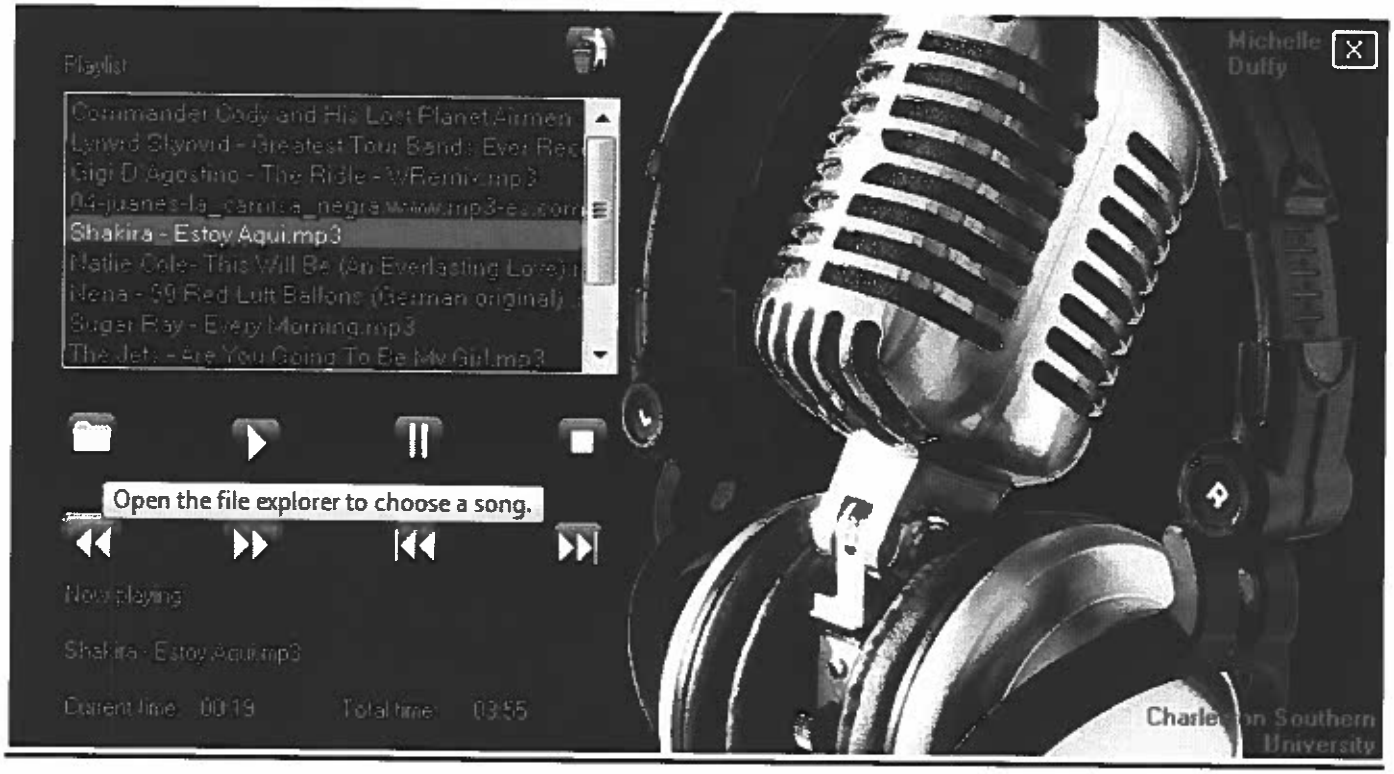


Song is stopped

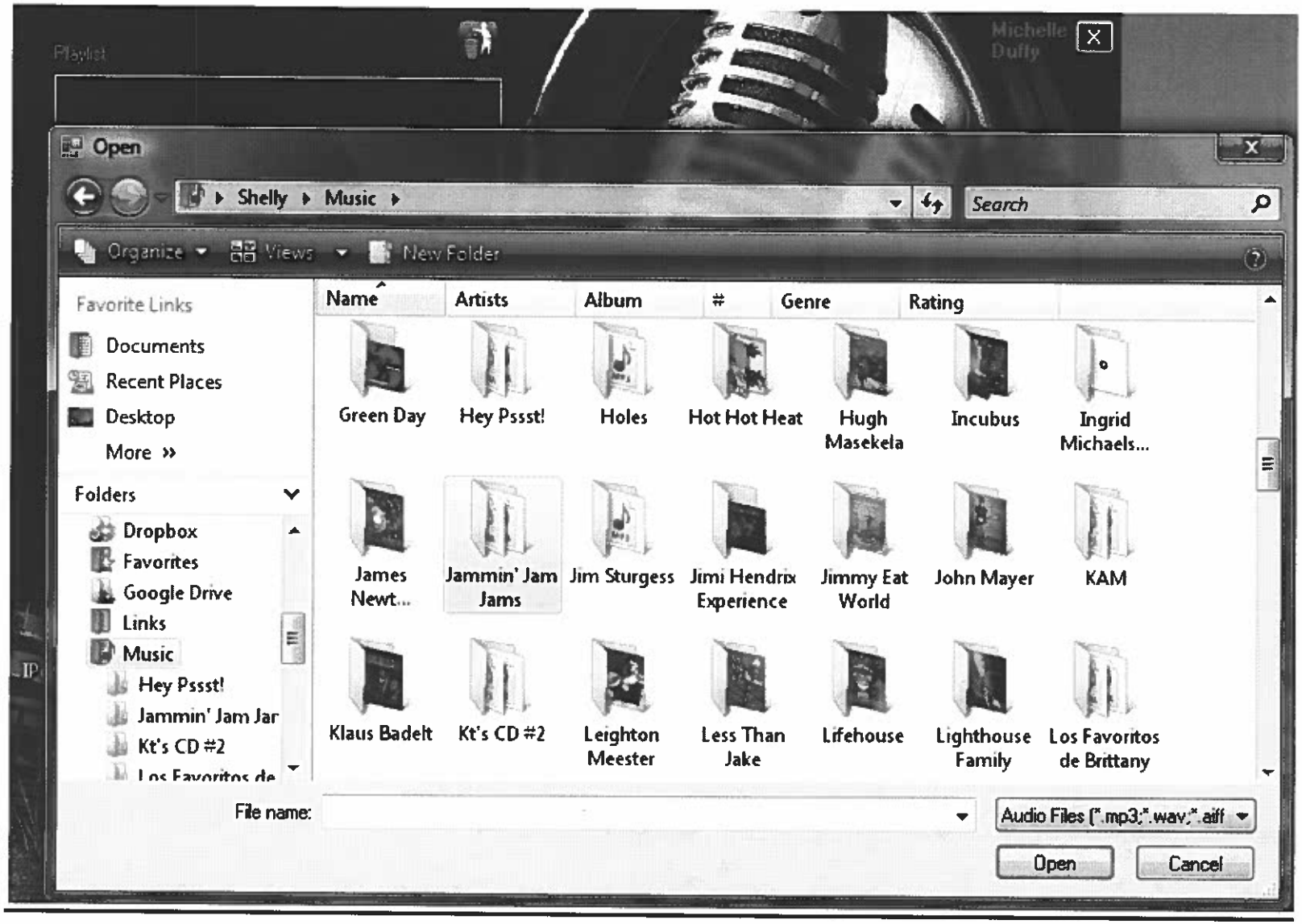
Player on open with an empty playlist

Error message if press Play Button when no song is selected

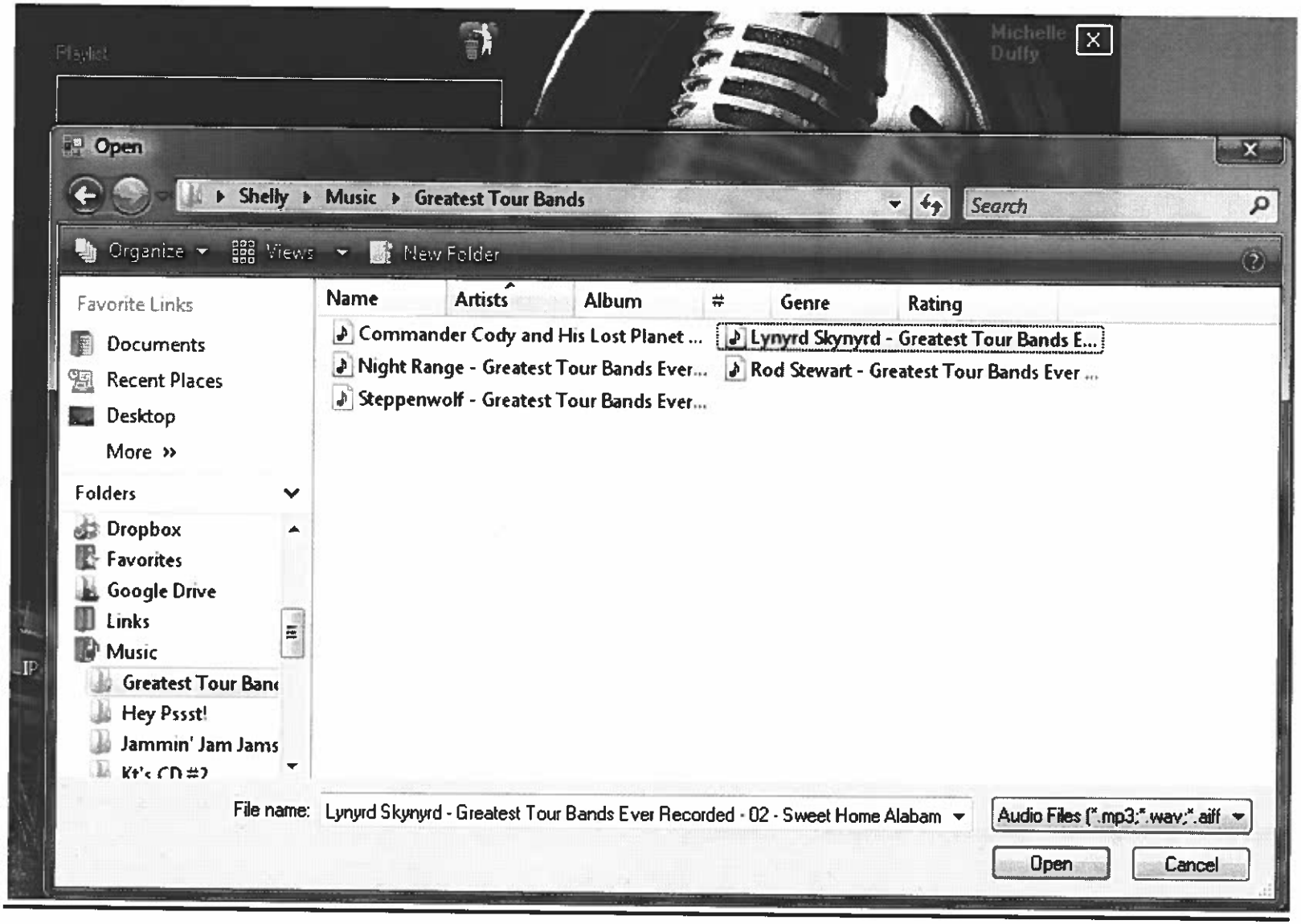
Tool tip for the Open Button



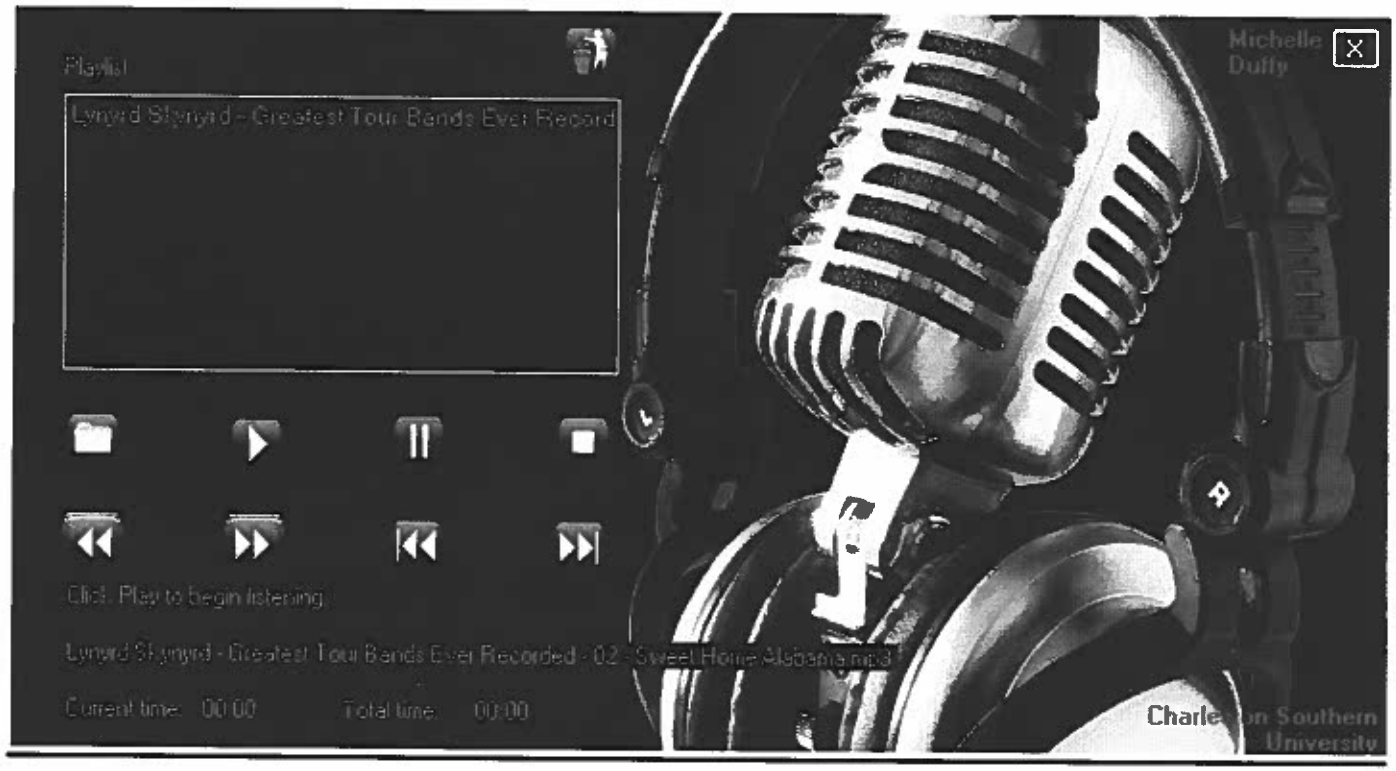
Open dialog-box



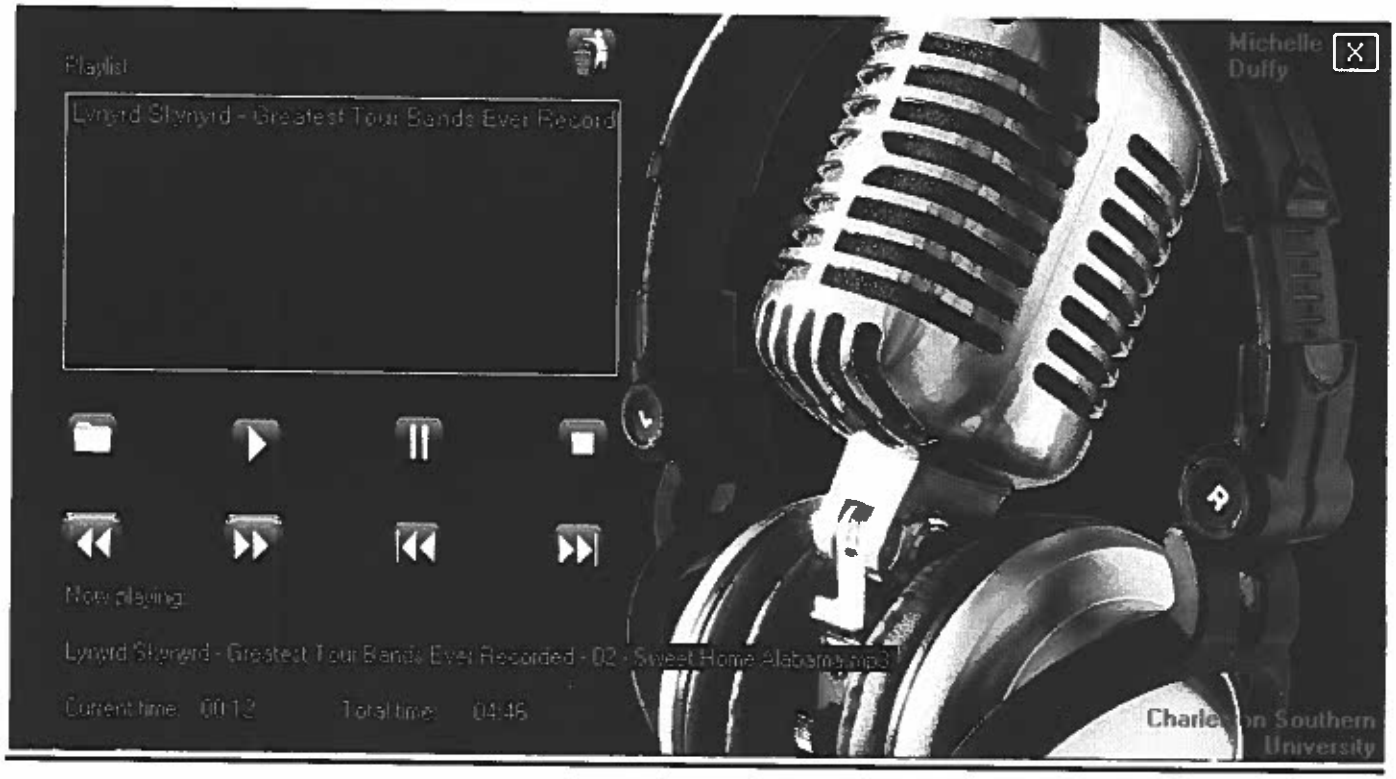
Open dialog-box with a song selected



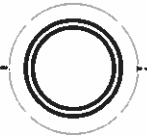
Song from the open dialog-box is ready to play



Playing a song from the open dialog-box



Ritmo: User Guide



MICHELLE DUFFY

SENIOR PROJECT

SUMMER 2014

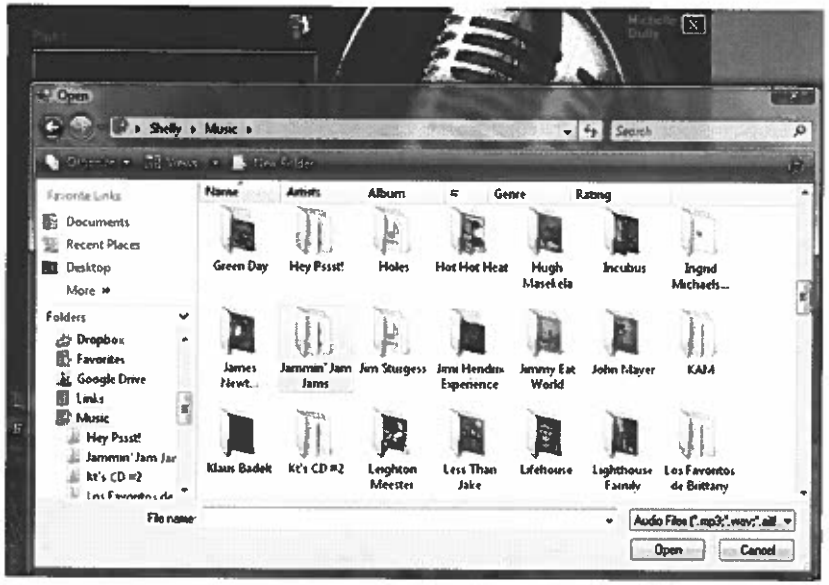
The Controls

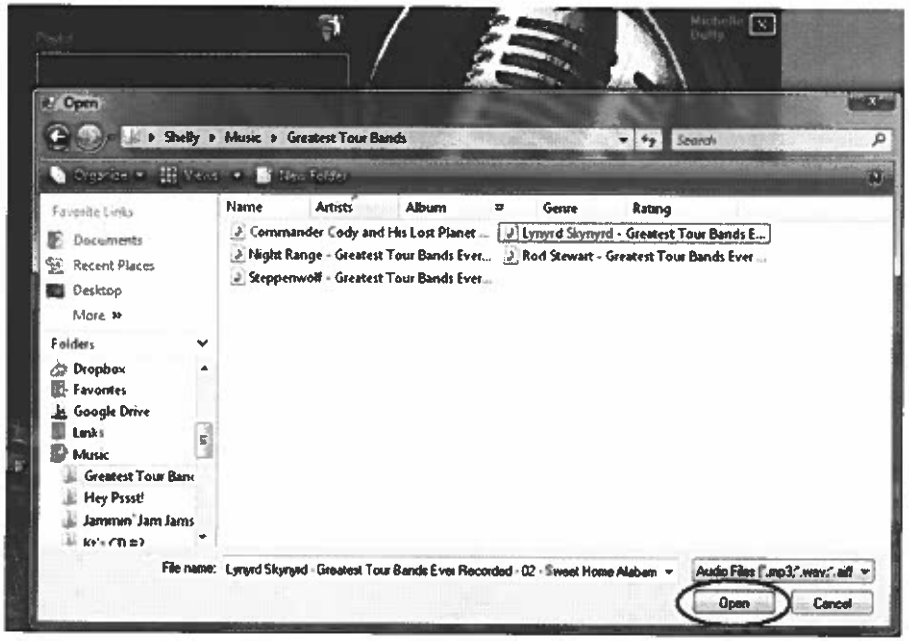
- Open File---------- 
- Play---------- 
- Pause---------- 
- Stop---------- 
- Rewind---------- 
- Fast Forward---------- 
- Previous Song---------- 
- Next Song---------- 
- Remove Song from Playlist---------- 

To Add a Song to the Playlist

Click the open button.

Select an audio file and click Open.





The title of the selected song will appear in the playlist.

Click Play to begin listening.

To Play a Song

Click the Play button.

You can also double-click the song title inside the playlist.

To Pause a Song

While a song is playing, click the Pause button.

Click the Play button to resume listening.

To Stop a Song

While a song is playing, click the Stop button.

Click the Play button to begin listening from the beginning of the song.

To Rewind a Song

While a song is playing, click the Rewind button.

To Fast Forward a Song

While a song is playing, click the Fast Forward button.

To Go to the Previous Song

Click the Previous button.

Click the Play button to begin listening.

To Go to the Next Song

Click the Next button.

Click the Play button to begin listening.

To Remove a Song from the Playlist

Select the song that you wish to be removed from the playlist.

Click the Remove from Playlist button.